



AnAxiomaticConcurrenceModel



S  +

S



-

R  +

R



—



S



—



C



—

R  +

R  -



C



—





po

po







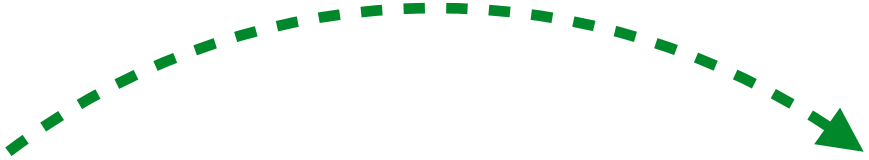
PO

po

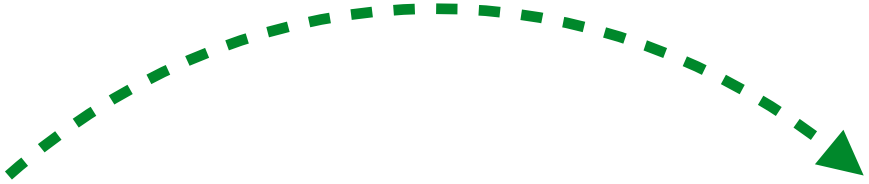


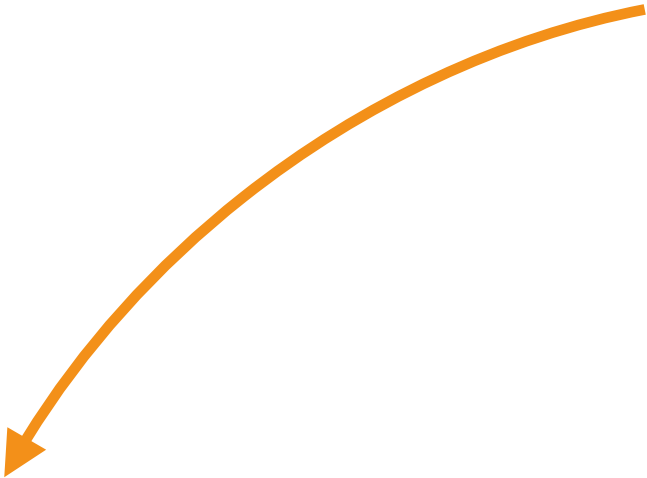
po

po

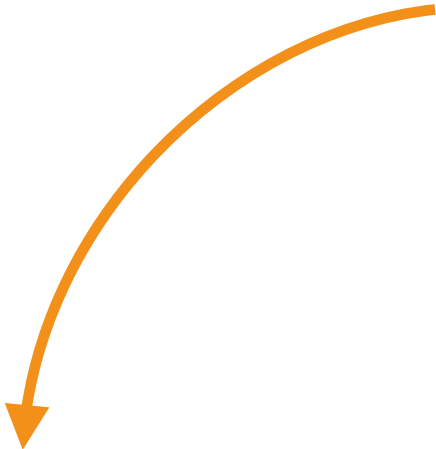




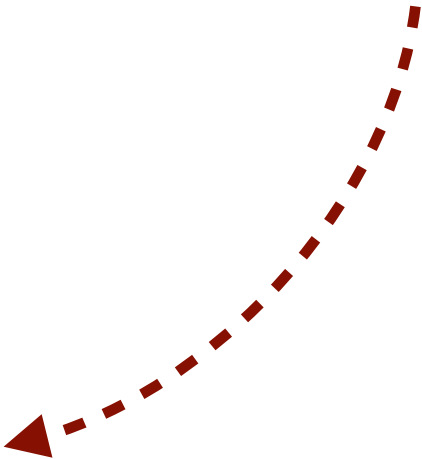


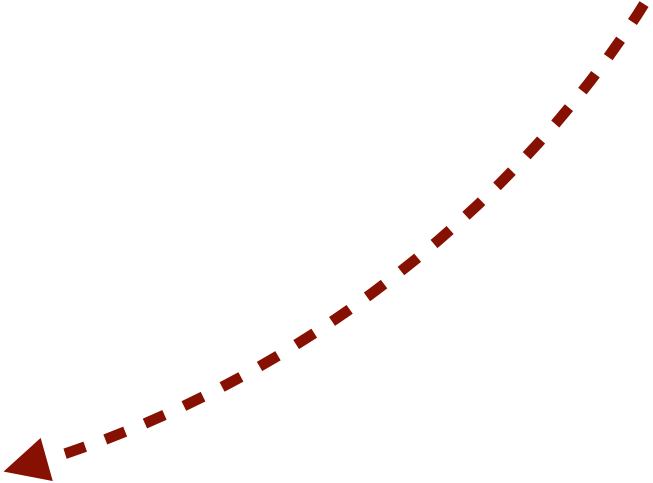


cosrc



collog





hb

h

b


```
update(src, +100)
```

```
update(src, -100)
```



Valid executions are acyclic in the transitive closure of the union relation

```
def update(account: cown[Account], amount: int) {  
  when(account) { A  
    if(account.balance + amount > 0) {  
      account.balance += amount  
      val id = account.id()  
      when(log) { B log.record(id, amount) }  
    } } }  
}
```

R  +

R



-







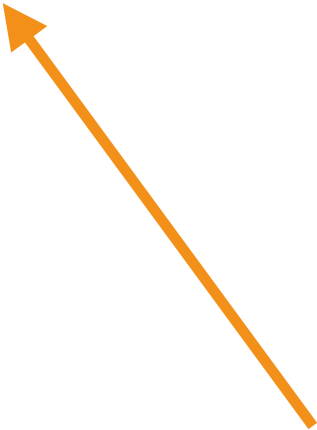
po

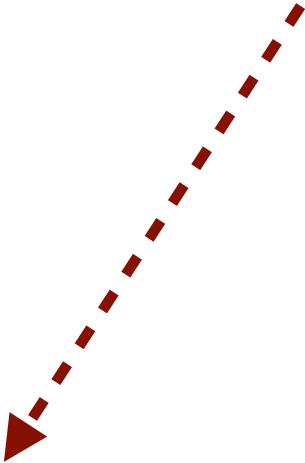
PO



collog







h

b







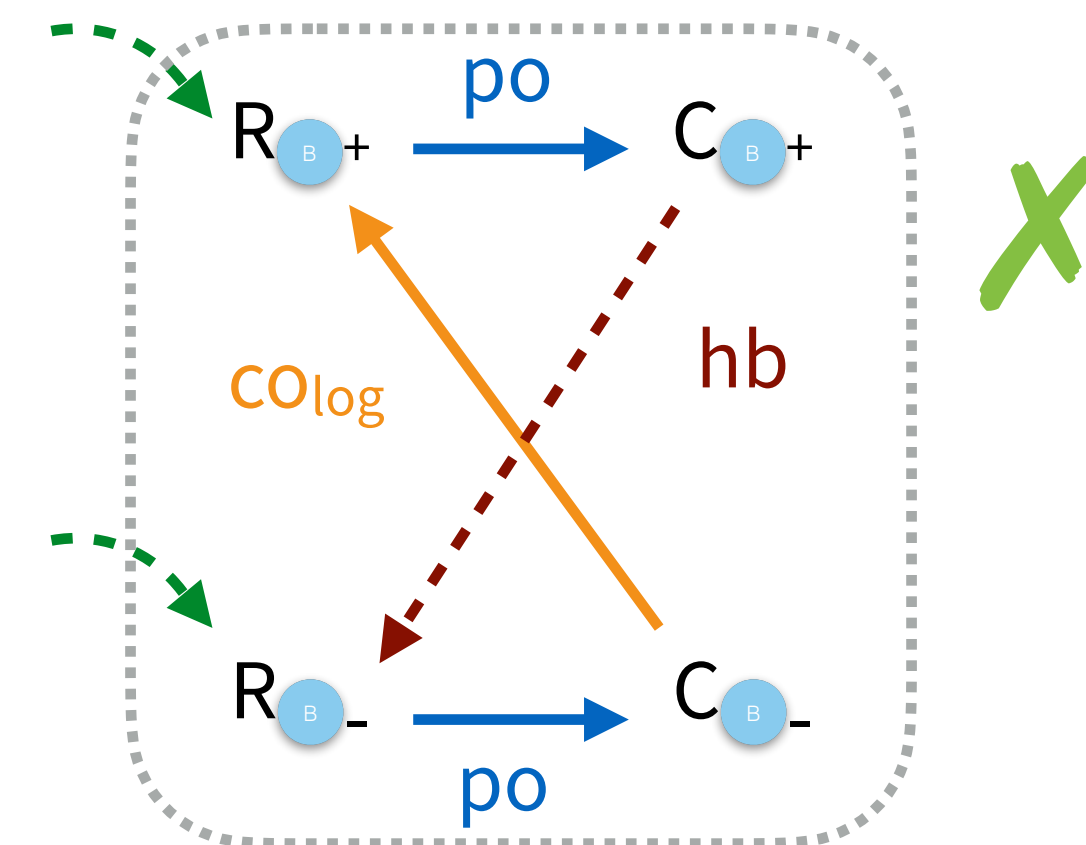
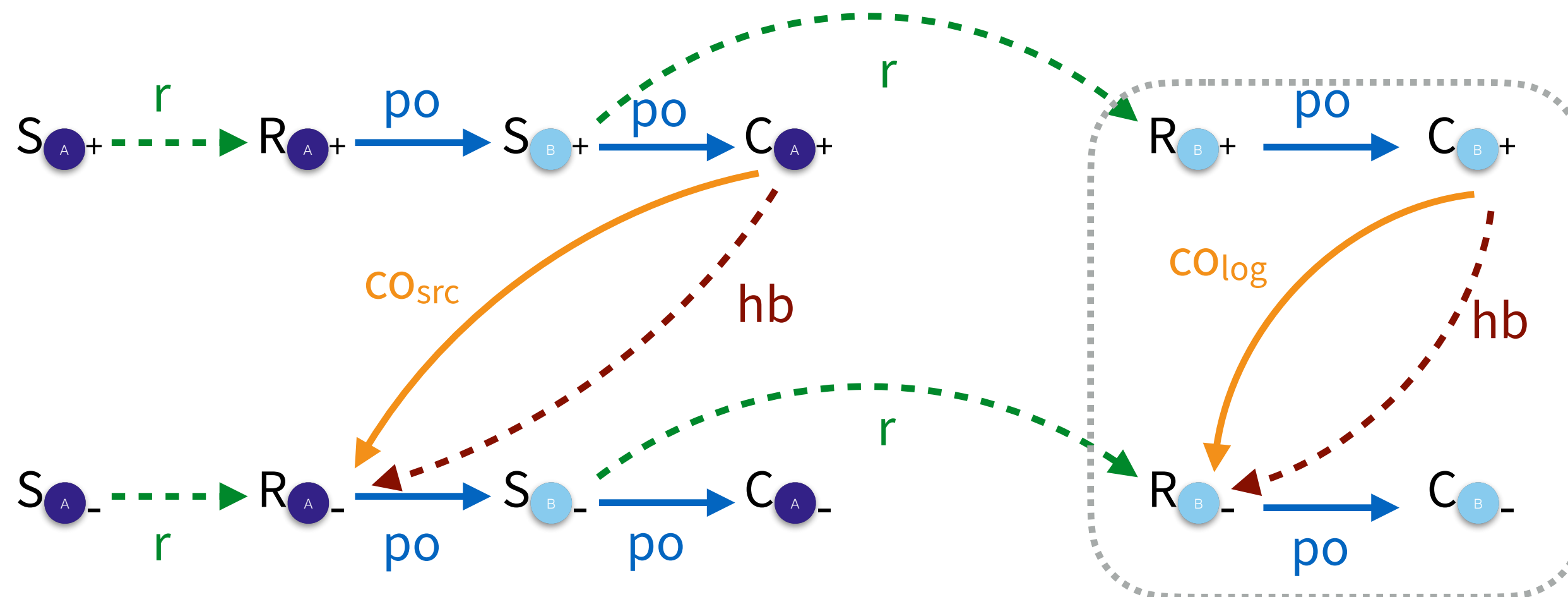
An Axiomatic Concurrency Model

```
def update(account: cown[Account], amount: int) {  
  when(account) { A  
    if(account.balance + amount > 0) {  
      account.balance += amount  
      val id = account.id()  
      when(log) { B log.record(id, amount) }  
    }  
  }  
}
```

update(src, +100)

update(src, -100)

Valid executions are acyclic in the transitive closure of the union of relations





What does this get us?

- What does it mean to say all of the executions the semantics provide are even correct?